

Employee Onboarding API — Project Overview

Generated by Claude Code
2026-07-20

Employee Onboarding API — Project Overview

1. What This Project Is

A small Spring Boot REST API for managing employee records as part of an onboarding system. It exposes CRUD-style endpoints for creating, listing, retrieving, filtering, and updating employees, backed by an in-memory H2 database. The project is used as a proof-of-concept for an AI-assisted development workflow (Claude Code driving a Jira → branch → TDD → PR → Confluence pipeline), so its size is intentionally small, but the engineering practices (TDD, layered architecture, exception handling) are production-style.

2. Tech Stack

Concern	Choice
Language	Java 21
Framework	Spring Boot 4.0.5
Persistence	Spring Data JPA + H2 (in-memory, jdbc:h2:mem:testdb)
API docs	springdoc-openapi (Swagger UI at /swagger-ui/index.html)
Boilerplate reduction	Lombok (@Data, @NoArgsConstructor, @AllArgsConstructor)
Testing	JUnit 5, Mockito, AssertJ, Spring MockMvc, @DataJpaTest, @SpringBootTest
Coverage	JaCoCo (Maven plugin, runs on mvn test)
Code quality	SonarQube (local Docker instance, localhost:9000)
Build tool	Maven
Issue tracking	Jira Cloud (nitinmvs-v2.atlassian.net)
Documentation	Confluence Cloud (nitinmvs-v2.atlassian.net/wiki)
Version control	Git + GitHub (Iris-user/employee-onboarding)

3. Project Structure

```
employee-onboarding/  
├── pom.xml  
├── README.md  
└── .claude/
```

```

├── CLAUDE.md # project workflow rules for Claude Code
├── src/main/java/com/example/
│   ├── Main.java # Spring Boot entry poin
│   └── employeeonboarding/
│       ├── model/Employee.java # JPA entity
│       ├── repository/EmployeeRepository.java # Spring Data repository
│       ├── service/EmployeeService.java # business logic
│       ├── controller/EmployeeController.java # REST endpoints
│       └── exception/
│           ├── EmployeeNotFoundException.java # 404
│           └── LastNameUpdateNotAllowedException.java # 400
├── src/main/resources/application.yml # DB / server config
├── src/test/java/com/example/
│   ├── MainTest.java # context-loads smoke t
│   └── employeeonboarding/
│       ├── EmployeeIntegrationTest.java # full-stack, real H2,
│       ├── repository/EmployeeRepositoryTest.java # @DataJpaTest
│       ├── service/EmployeeServiceTest.java # Mockito unit tests
│       └── controller/EmployeeControllerTest.java # @WebMvcTest, mocked
└── service

```

This is a classic layered architecture: **Controller** → **Service** → **Repository** → **Database**, with a separate exception package for domain-specific errors.

4. Domain Model

Employee (model/Employee.java) is a JPA entity with an auto-generated identity id and four plain fields: name, lastName, email, department. Lombok's @Data generates getters/setters>equals/hashCode/toString, and both a no-args and all-args constructor are generated for JPA and test convenience.

5. Data Access Layer

EmployeeRepository extends Spring Data's JpaRepository<Employee, Long>, which provides save, findAll, findById, deleteAll, etc. for free. One derived query method is added: findByDepartment(String department), which Spring Data implements automatically from the method name (translates to SELECT * FROM employee WHERE department = ?).

6. Service Layer (Business Logic)

EmployeeService wraps the repository and adds the actual business rules:

- **save(Employee)** — persists a new employee as-is.
- **getAll()** — returns every employee.

- **getById(Long id)** — returns the employee or null if not found (the controller translates null into a 404).
- **getByDepartment(String department)** — delegates to the derived repository query.
- **updateLastName(Long id, String lastName)** — looks up the employee, throws a generic RuntimeException if missing, otherwise updates and saves.
- **update(Long id, Employee updatedEmployee)** — the most interesting method:
 - Throws EmployeeNotFoundException (→ HTTP 404) if the ID doesn't exist.
 - **Deliberately rejects changes to lastName** through this endpoint: if the incoming payload's lastName differs from the stored value, it throws LastNameUpdateNotAllowedException (→ HTTP 400). Last name can *only* be changed via the dedicated PATCH /employees/{id}/last-name endpoint.
 - Otherwise applies name, email, and department changes and saves.

This split exists so that “last name” changes go through an explicit, auditable path (the PATCH endpoint) rather than being silently overwritten as part of a general update — a deliberate business rule, not an oversight.

7. REST API (Controller Layer)

All endpoints are under EmployeeController, mapped to base path /employees:

Method	Path	Description	Success	Failure
POST	/employees	Create an employee	200 OK	—
GET	/employees	List all employees	200 OK	—
GET	/employees/{id}	Get one employee by ID	200 OK	404 if missing
GET	/employees/department/{dept}	List employees filtered by department	200 OK (possibly empty array)	—
PUT	/employees/{id}	Update name/email/department (not lastName)	200 OK	404 if missing,

Method	Path	Description	Success	Failure
				400 if last Name is changed
PATCH	/employees/{id}/last-name	Update only lastName	200 OK	— (currently throws unhandled RuntimeException on missing ID, see §9)

Example request body for POST/PUT:

```
{
  "name": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "department": "Engineering"
}
```

Example request body for PATCH .../last-name:

```
{ "lastName": "Smith" }
```

8. Error Handling

Two custom exceptions map to HTTP status codes via `@ResponseStatus`:

- `EmployeeNotFoundException` → **404 Not Found**
- `LastNameUpdateNotAllowedException` → **400 Bad Request**

Both are thrown from `EmployeeService.update()`. There's no `@ControllerAdvice/global` exception handler — Spring's default `@ResponseStatus` mechanism on the exception classes themselves is what maps them to HTTP status codes.

9. Known Inconsistency Worth Flagging

`EmployeeService.updateLastName()` still throws a plain `RuntimeException` (message: "Employee not found with id: " + id) instead of the newer `EmployeeNotFoundException` used by `update()`. Functionally this means a missing-ID PATCH to `/last-name` will currently surface as a generic 500 Internal Server Error rather than a clean 404, inconsistent with the PUT endpoint's behavior. This isn't called out in any Jira ticket seen so far — worth a follow-up ticket if consistent error semantics matter.

10. Configuration

`src/main/resources/application.yml`: - Datasource: in-memory H2, `jdbc:h2:mem:testdb` — **data does not persist across restarts**, this is a demo/dev setup, not production-ready storage. - JPA: `ddl-auto: update` (schema auto-created/updated from entities), SQL logging enabled (`show-sql: true`). - H2 web console enabled at `/h2-console` for inspecting the in-memory DB during development. - Server runs on port 8080.

11. Testing Strategy

Four distinct test layers, all JUnit 5:

1. **EmployeeServiceTest** — pure unit tests, repository mocked with Mockito. Covers every service method including both branches of the last-name-change guard in `update()`.
2. **EmployeeControllerTest** — `@WebMvcTest`, only the web layer loaded, service mocked via `@MockitoBean`. Verifies HTTP status codes and JSON shape for every endpoint, including the 400/404 error paths.
3. **EmployeeRepositoryTest** — `@DataJpaTest`, real (embedded) database, verifies the derived `findByDepartment` query actually filters correctly.
4. **EmployeeIntegrationTest** — `@SpringBootTest + @AutoConfigureMockMvc`, full application context, exercises the whole stack end-to-end through real HTTP-style requests against a real (in-memory) database.

Plus `MainTest`, a smoke test that boots the full Spring context via the actual `main()` method to confirm the application starts.

Coverage is measured by JaCoCo, which is wired into `mvn test` via `prepare-agent/report` executions in `pom.xml`. `Main.java` is explicitly excluded from Sonar coverage requirements (`sonar.coverage.exclusions: **/Main.java`), since it's just the framework bootstrap.

12. Build, Run, Test

```
mvn clean install          # build
mvn spring-boot:run       # run locally on :8080
mvn test                  # run the test suite (+ JaCoCo coverage report)
mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.token=<token>
    # quality scan
```

Swagger UI: <http://localhost:8080/swagger-ui/index.html> OpenAPI spec:
<http://localhost:8080/v3/api-docs> H2 console: <http://localhost:8080/h2-console>

13. Development Workflow (per `.claude/CLAUDE.md`)

The repo has a defined Claude-Code-driven workflow for feature work, given a Jira ID:

1. Fetch the Jira ticket via MCP.
2. Pull latest main, create `feature/<JIRA-ID>-<short-description>` branch.
3. Move the Jira ticket to **In Progress**.
4. TDD: write a failing JUnit 5 test first.
5. Confirm it fails, then implement the feature.
6. Run the SonarQube scan and fix any reported issues/code smells (formalized as the `/sonar-scan` skill, see `.claude/skills/sonar-scan/SKILL.md`).
7. Commit as `<JIRA-ID>: <ticket summary>`, push, open a PR (never merge — PR only).
8. Update `README.md`.
9. Comment on the Jira ticket with the change summary + PR link, move it to **In Review**.

Rules explicitly enforced: no Python, no pip/virtualenvs, no Node.js — Maven only for build/test/dependency management; never push directly to main.

14. Jira & Confluence Integration

- **Jira:** Cloud site `nitinmvs-v2.atlassian.net`, accessed via the atlassian MCP server (Docker MCP gateway). Used to track features as tickets (e.g. TC-2 through TC-8 seen so far, covering get-by-id, update, department filter, and last-name/PATCH features).
- **Confluence:** same Atlassian site, `/wiki` path, same MCP server. A **“TC Board JIRA”** folder holds one sub-page per completed ticket, documenting status, description, implementation summary, and PR link — kept in sync with Jira as tickets complete.

15. Git / Repository State

- Remote: <https://github.com/Iris-user/employee-onboarding.git>
- Current branch (at time of writing): `feature/TC-4-employees-by-department`

- Recent history shows incremental ticket-by-ticket delivery: TC-2 (get by ID), TC-3 (update by ID), TC-4 (department filter), TC-8 (last name + PATCH, retroactively ticketed for PR #6).
- Branch naming convention: feature/<JIRA-ID>-<short-description>.

16. Summary

This is a compact, TDD-built Spring Boot CRUD API for employee records, notable less for its size and more for the surrounding engineering discipline: every feature maps to a Jira ticket, is developed test-first on its own branch, gated by a local SonarQube scan, delivered via PR only (no direct merges to main), and documented back into both the README and a mirrored Confluence hierarchy. The one functional gap worth addressing is the inconsistent error handling in `updateLastName` (§9).